SPRING BOOT ANNOTATION NOTES

1. INTRODUCTION TO ANNOTATIONS

→ Annotations are metadata that provide information about classes, methods, variables or configuration.

→ They don't change the logic, but they tell the framework what to do.

→ Spring Boot and Hibernate heavily use annotations to reduce XML configuration and make development faster.

KEY POINT

Annotations make our code clean, readable and easy to maintain.

HOW SPRING BOOT WORKS (HIGH LEVEL FLOW)

Client (Browser / App) → Spring Boot Application → Spring Container (IoC Container) → Beans Created & Managed → Database / External Systems

WHY ANNOTATIONS?

- ✓ Reduce boilerplate code
- ✓ Easy configuration
- ✓ Loose coupling
- ✓ Improved readability
- ✓ Better maintainability
- ✓ Faster development

BEFORE vs AFTER USING ANNOTATIONS

Before (XML Based)

- Lots of XML configuration
- Hard to read and maintain
- More setup time

After (Annotation Based)

- Simple annotations
- Clean and readable code
- Less configuration
- Faster development

TYPES OF ANNOTATIONS IN SPRING BOOT (OVERVIEW)

Type	Purpose	Example Annotations
Stereotype Annotations	Define the role of a class in the Spring container.	@Component, @Service, @Repository, @Controller, @RestController
Bean & DI Annotations	Handle bean creation and dependency injection.	@Autowired, @Qualifier, @Primary, @Bean, @Value
Configuration Annotations	Used to define configuration and application behavior.	@Configuration, @ComponentScan, @PropertySource, @Value, @ConfigurationProperties
Web & Request Annotations	Used for handling web requests and responses.	@Controller, @GetMapping, @PostMapping, @RequestParam, @PathVariable, @RequestBody
Data & JPA Annotations	Used for database mapping and JPA operations.	@Entity, @Table, @Id, @Column, @OneToMany, @ManyToOne, @Transactional
Others	Used for additional features like scheduling, caching, etc.	@Async, @Scheduled, @Cacheable, @EventListener, @Profile

☆ **NOTE:**

In this series, we will cover all important Hibernate & Spring Boot annotations with examples, diagrams and use cases.

→ Let's start with Stereotype Annotations on next page.

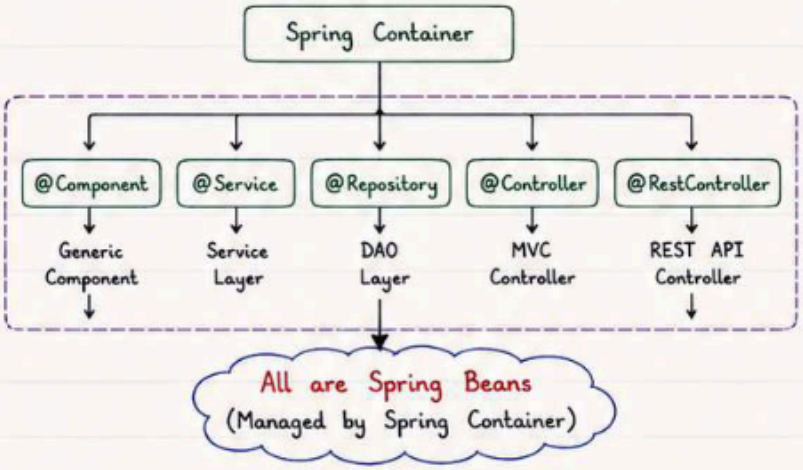
2. CORE STEREOTYPE ANNOTATIONS

→ These annotations are used to declare the role of a class in the Spring container.

Note: Spring scans these annotations and creates bean automatically.

Annotation	Purpose	When to use?	Example
@Component	Generic stereotype for any Spring-managed component.	When a class doesn't fit in specific categories.	`@Component` `public class MyComponent { }`
@Service	Used for service layer classes.	Business logic or service layer components.	`@Service` `public class UserService { }`
@Repository	Used for DAO layer. It also enables exception translation.	Data access layer (DB operations).	`@Repository` `public class UserRepository { }`
@Controller	Used for traditional MVC web applications.	When returning view names (HTML pages).	`@Controller` `public class HomeController { }`
@RestController	Combination of @Controller + @ResponseBody.	REST APIs (returns JSON instead of view).	`@RestController` `public class UserController { }`

HOW THEY ARE RELATED?



KEY DIFFERENCES

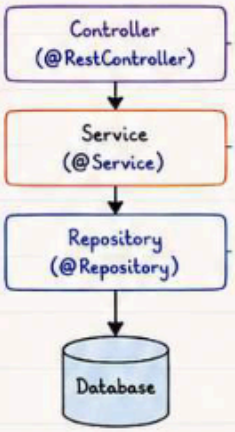
Annotation	Key Difference
@Component	Generic purpose, no specific role.
@Service	Business logic layer.
@Repository	Data access layer + exception translation.
@Controller	Handles web requests and returns views.
@RestController	Handles web requests and returns JSON (REST APIs).

EXAMPLE - SIMPLE LAYERED APPLICATION

```
// Controller Layer
@RestController
public class UserController {
}

// Service Layer
@Service
public class UserService {
}

// Repository Layer
@Repository
public interface UserRepository extends JpaRepository<User, Long> {
}
```



Pro Tip:
In most projects, you will use @Service, @Repository and @RestController most of the time.

3. DEPENDENCY INJECTION ANNOTATIONS

→ These annotations help Spring create and inject the dependencies (beans) automatically.

Key Idea

Spring creates objects, manages their lifecycle and injects dependencies instead of us creating manually.

A. @Autowired

- Used to inject dependencies automatically.
- By default, it uses byType strategy.
- Can be used on fields, constructors and setters.

```
@Service
public class UserService {
    @Autowired
    private UserRepository userRepository;
}
```

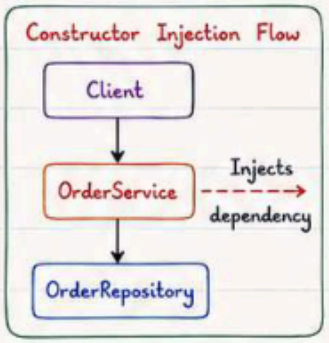
--→ Spring will inject the bean of type UserRepository automatically.

B. Constructor Injection (Recommended)

- ✓ Mandatory dependencies
- ✓ Immutable
- ✓ Easy to test
- ✓ Preferred over field injection

```
@Service
public class OrderService {
    private final OrderRepository orderRepository;

    @Autowired
    public OrderService(OrderRepository orderRepository) {
        this.orderRepository = orderRepository;
    }
}
```



C. Setter Injection

- Used when the dependency is optional or can be changed.
- Not preferred for mandatory dependencies.

```
@Service
public class PaymentService {
    private PaymentGateway gateway;

    @Autowired
    public void setGateway(PaymentGateway gateway) {
        this.gateway = gateway;
    }
}
```

When to use?

- Optional dependencies
- Changing dependencies at runtime

D. @Qualifier

- Used when multiple beans of same type exist.
- Helps to specify which bean should be injected.

```
@Service
public class EmailService {
    private final EmailSender emailSender;

    @Autowired
    public EmailService(@Qualifier("smtpEmailSender")
                        EmailSender emailSender) {
        this.emailSender = emailSender;
    }
}
```

Example

```
@Component
@Qualifier("smtpEmailSender")
public class SMTPEmailSender
implements EmailSender { }
```

```
@Component
@Qualifier("sesEmailSender")
public class SESEmailSender
implements EmailSender { }
```

↳ @Qualifier helps Spring choose the right bean.

E. @Primary

- Used to mark a primary bean when multiple beans of same type exist.
- It will be chosen by default if @Qualifier is not used.

```
@Component
@Primary
public class DefaultEmailSender implements EmailSender { }

@Component
public class BackupEmailSender implements EmailSender { }
```

Note

If a bean is marked as @Primary, Spring will inject it by default when multiple beans are available.

4. BEAN CREATION & CONFIGURATION ANNOTATIONS

→ These annotations are used to define beans, configuration classes and control component scanning in Spring Boot.

Key Idea

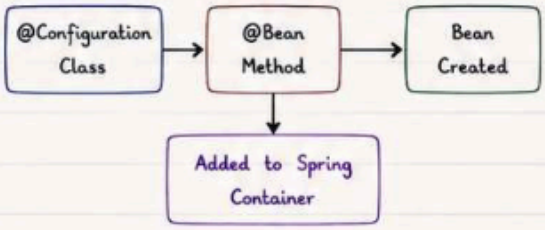
Spring Boot uses these annotations to create beans, load configuration and manage the Spring container.

A. @Bean

- Used to declare a bean explicitly.
- Method annotated with @Bean returns the object to be managed by Spring.
- Often used inside @Configuration classes.

```
@Configuration
public class AppConfig {
    @Bean
    public DataSource dataSource() {
        return new HikariDataSource();
    }
}
```

How it works?



B. @Configuration

- Marks a class as a source of bean definitions.
- Indicates that the class contains @Bean methods.
- Acts like an XML configuration file.

```
@Configuration
public class DatabaseConfig {
    @Bean
    public DataSource dataSource() {
        return new HikariDataSource();
    }
}
```

Important Notes

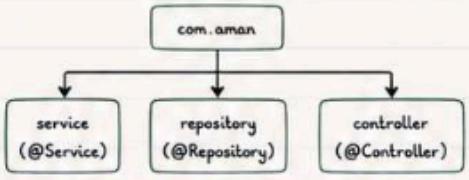
- ✓ @Configuration classes are processed by Spring at startup.
- ✓ @Bean methods produce and manage beans.

C. @ComponentScan

- Tells Spring where to look for components (@Component, @Service, @Repository, etc.).
- By default, it scans the package of the main class.

```
@SpringBootApplication
@ComponentScan(basePackages = {"com.aman.service",
                               "com.aman.repository"})
public class MyApplication {
    public static void main(String[] args) {
        SpringApplication.run(MyApplication.class, args);
    }
}
```

Example Structure



All these packages will be scanned and registered as beans.

D. @Import

- Used to import additional configuration classes.
- Useful when configuration is split across multiple classes.
- Helps in modular configuration.

```
@Configuration
public class AppConfig {
    @Bean public String appName() { return "AmanApp"; }
}

@Configuration
public class DBConfig {
    @Bean public String dbName() { return "AmanDB"; }
}
```

```
@SpringBootApplication
@Import({AppConfig.class, DBConfig.class})
public class MyApplication {
    public static void main(String[] args) {
        SpringApplication.run(MyApplication.class, args);
    }
}
```

@Import tells Spring to load the beans from the specified configuration classes.

E. @Lazy

- Instructs Spring to create the bean only when it is needed.
- Helps in improving startup performance.
- Can be used on classes or @Bean methods.

On Class

```
@Service
@Lazy
public class ReportService {
    // bean will be created when
    // first time requested
}
```

On @Bean Method

```
@Configuration
public class AppConfig {
    @Bean
    @Lazy
    public ExpensiveService service() {
        return new ExpensiveService();
    }
}
```

When to use?

- ✓ Heavy beans
- ✓ Beans with costly initialization
- ✓ Optional features
- ✓ To avoid ciruclar dependency problems at startup

5. CONFIGURATION ANNOTATIONS

→ These annotations are used to externalize configuration, manage properties and enable profile based configurations.

Key Idea

They help us to make application configurable, flexible and environment specific.

A. @Value

- Used to inject values from properties files.
- Works with ${...} expression.
- Can be used on fields, constructors and methods.

```
@Component
public class AppInfo {
    @Value("${app.name}")
    private String appName;

    @Value("${server.port}")
    private int port;
}
```

application.properties

```
app.name = AmanApp
server.port = 8080
app.version = 1.0
```

Use @Value when you need a single property value.

B. @ConfigurationProperties

- Used to bind a group of properties to a POJO.
- Reduces boilerplate code.
- Supports type-safe configuration.
- Works with prefix.

```
@Component
@ConfigurationProperties(prefix = "app")
public class AppProperties {
    private String name;
    private String version;
    private String owner;

    // getters and setters
}
```

application.yml

```
app:
  name: AmanApp
  version: 1.0
  owner: Aman Saini
```

Use @ConfigurationProperties for multiple related properties.

C. @PropertySource

- Used to load properties from an external file.
- By default, Spring loads application.properties or application.yml.
- Use this annotation for custom property files.

```
@Configuration
@PropertySource("classpath:custom.properties")
public class AppConfig {

    // beans
}
```

custom.properties

```
custom.key = customValue
custom.message = Hello from
    custom properties file!
```

Use when you have separate properties files.

D. @Profile

- Used to define environment specific beans.
- Activate only for a specific profile.
- Useful for dev, test, prod environments.

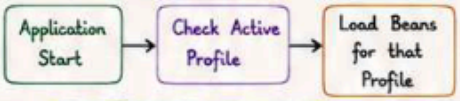
```
@Configuration
@Profile("dev")
public class DevConfig {
    @Bean
    public DataSource dataSource() {
        // dev datasource
    }
}
```

```
@Configuration
@Profile("prod")
public class ProdConfig {
    @Bean
    public DataSource dataSource() {
        // prod datasource
    }
}
```

How to activate profile?

1. application.properties
   spring.profiles.active = dev
2. VM Option
   -Dspring.profiles.active=prod

Profile Flow



Helps in maintaining different configurations for different environments.

NOTE:

- ✓ Prefer @ConfigurationProperties over @Value for multiple properties.
- ✓ Use @Profile to keep environment specific configurations.
- ✓ Externalize configuration to make your application flexible and production ready.

Pro Tip:

Keep your sensitive data in environment variables and access using @Value.

6. WEB & REQUEST MAPPING ANNOTATIONS

→ These annotations are used in Spring MVC / Spring Boot to map HTTP requests to controller methods.

Key Idea

They help Spring understand which URL / HTTP method should call which method.

A. Class Level vs Method Level Mapping

1. Class Level (@RequestMapping)

- Base path for all methods
- Avoids repeating common path

```
@RestController
@RequestMapping("/api/users")
public class UserController {
    // all methods will start with /api/users
}
```

÷ 2. Method Level Mapping

- Maps specific HTTP methods (GET, POST, etc.)
- Handles specific endpoints

```
@GetMapping("/{id}")
public User getUser(@PathVariable Long id) {
    // logic
}
```

Request Flow

Client Request (URL + HTTP Method)
↓
DispatcherServlet
↓
Handler Mapping (Finds Controller Method)
↓
Controller Method Executed
↓
Response Returned (JSON / View)

B. Common Mapping Annotations (Method Level)

Annotation	HTTP Method	Purpose	Example
@GetMapping	GET	Fetch data	@GetMapping("/users")
@PostMapping	POST	Create new resource	@PostMapping("/users")
@PutMapping	PUT	Update full resource	@PutMapping("/users/{id}")
@DeleteMapping	DELETE	Delete resource	@DeleteMapping("/users/{id}")
@PatchMapping	PATCH	Update partial resource	@PatchMapping("/users/{id}")

C. Other Useful Annotations

Annotation	Use
@PathVariable	Extract value from URL path
@RequestParam	Extract query parameters (?id=1)
@RequestBody	Extract JSON data from request body
@ResponseBody	Return response directly (usually JSON)

D. PathVariable vs RequestParam

@PathVariable
- Used in URL path
- Mandatory
- Example: /users/101
  @GetMapping("/users/{id}")

VS

@RequestParam
- Used in query parameter
- Optional (by default)
- Example: /users?id=101
  @GetMapping("/users")

Example - Complete

```
@RestController
@RequestMapping("/api/products")
public class ProductController {
    @GetMapping("/{id}")
    public Product getProduct(@PathVariable Long id) {
        // code
        return product;
    }

    @PostMapping
    public Product createProduct(@RequestBody Product product) {
        // code
        return product;
    }
}
```

NOTE:

✓ Always prefer @RestController for REST APIs.
✓ Use meaningful and consistent URLs.

Pro Tip:

Keep your URLs resource based.
Example: /users, /products, /orders
(Not: /getUsers, /fetchProduct)

7. DATA ACCESS & TRANSACTION ANNOTATIONS

→ These annotations simplify database access and manage transactions in Spring Boot.

Key Idea

Spring provides annotations to reduce boilerplate code for data access and handle transactions declaratively.

A. DATA ACCESS ANNOTATIONS

Annotation	Purpose	Example
@Repository	Marks class as Repository (Data Access Layer)	@Repository public class UserRepository {}
@Entity	Marks class as JPA Entity	@Entity public class User {}
@Table	Specifies table name for Entity	@Table(name = "users")
@Id	Marks primary key field	@Id private Long id;
@GeneratedValue	Auto generation strategy for primary key	@GeneratedValue(strategy = GenerationType.IDENTITY)
@Column	Specifies column details	@Column(name = "user_name", nullable = false, length = 50)
@Transient	Excludes field from database mapping	@Transient private String tempData;

Example – Entity Class

```
@Entity
@Table(name = "users")
public class User {
   @Id
   @GeneratedValue(strategy = GenerationType.IDENTITY)
   private Long id;

   @Column(name = "user_name", nullable = false, length = 50)
   private String name;

   @Column(unique = true)
   private String email;

   @Transient
   private String tempData;
}
```

Note:
@Entity + @Table + @Id are the most important combination for JPA mapping.

B. TRANSACTION ANNOTATIONS

- ✓ Manage database transactions declaratively.
- ✓ Ensures data consistency.
- ✓ Works with Spring's PlatformTransactionManager.

Transaction Flow



Annotation	Purpose	Example / Use
@Transactional	Marks method/class as transactional	@Transactional public void createUser() { ... }
@Transactional(readOnly = true)	Optimized for read-only operations	@Transactional(readOnly = true) public List<User> getAllUsers() { ... }
@Transactional(rollbackFor = Exception.class)	Rollback for specific exceptions	@Transactional(rollbackFor = Exception.class) public void saveData() { ... }
@Transactional(propagation = Propagation.REQUIRES_NEW)	Starts a new transaction	@Transactional(propagation = Propagation.REQUIRES_NEW) public void auditLog() { ... }

Example – Transactional Service

```
@Service
public class UserService {
   @Autowired
    private UserRepository userRepository;

   @Transactional
    public void createUser(User user) {
        userRepository.save(user);
        // other business logic
    }
}
```

Class Level vs Method Level

1. Class Level
   - All public methods are transactional.
   - Can be overridden at method level.

```
@Service
@Transactional
public class UserService {
    // all public methods
    // transactional
}
```

2. Method Level
   - Only specific methods are transactional.
   - More flexible.

```
@Service
public class UserService {
    @Transactional
    public void createUser() { ... }
}
```

Best Practices

- ✓ Use @Transactional for write operations.
- ✓ Use readOnly = true for read operations.
- ✓ Handle exceptions properly for rollback.
- ✓ Keep transactions short and focused.
- ✓ Avoid calling external APIs inside transaction.

8. STEREOTYPE, COMPONENT & OTHER ANNOTATIONS

→ These annotations are useful for marking roles, defining components and handling special cases.

Key Idea

They help Spring understand the role of a class and make your application more modular and maintainable.

A. STEREOTYPE ANNOTATIONS

Annotation	Use	Example

@Component	Generic stereotype for any Spring managed component.	@Component
		public class EmailService {
		// ...
		}

@Service	Marks class as Service layer.	@Service
		public class UserService {
		// ...
		}

@Repository	Marks class as Repository (DAO layer).	@Repository
		public interface UserRepo
		extends JpaRepository<User, Long> {
		}

@Controller	Marks class as Spring MVC Controller.	@Controller
		public class UserController {
		// ...
		}

@RestController	@Controller + @ResponseBody (for REST APIs).	@RestController
		public class UserRestController {
		// ...
		}

B. COMPONENT SCANNING ANNOTATIONS

@ComponentScan

- Tells Spring where to look for components.
- By default scans current package and sub-packages.

Example :

```
@SpringBootApplication
@ComponentScan(basePackages = "com.example")
public class MyApplication {
  public static void main(String[] args) {
    SpringApplication.run(MyApplication.class, args);
  }
}
```

Note :

★ @SpringBootApplication already includes @ComponentScan.

★ Use @ComponentScan only when you want to scan specific packages.

C. OTHER USEFUL ANNOTATIONS

Annotation	Use	Example

@Configuration	Marks class as source of bean definitions.	@Configuration
		public class AppConfig {
		// bean definitions
		}

@Bean	Declares a method that returns a bean to be managed by Spring.	@Bean
		public DataSource dataSource() {
		return new HikariDataSource();
		}

@Scope	Defines scope of a bean (singleton, prototype, etc.).	@Scope("prototype")
		@Component
		public class MyBean {
		// ...
		}

@DependsOn	Specifies that a bean depends on other bean(s).	@Component
		@DependsOn("dataSource")
		public class MyService {
		} // ...

@Lazy	Initialize bean lazily (only when needed).	@Lazy
		@Component
		public class HeavyService {
		// ...
		}

D. QUICK SUMMARY

- ✓ Use @Component for generic components.
- ✓ Use @Service for service layer.
- ✓ Use @Repository for data access layer.
- ✓ Use @Controller / @RestController for web layer.
- ✓ @ComponentScan helps Spring find your beans.
- ✓ @Configuration + @Bean used to define beans manually.

Best Practices

- ✓ Prefer specific annotations over @Component.
- ✓ Keep components in logical packages.
- ✓ Avoid component scan on too broad packages.
- ✓ Use @Lazy for heavy or rarely used beans.
- ✓ Keep configuration classes clean and focused.

9. SPRING BOOT + PROFILE + PROPERTY ANNOTATIONS

→ These annotations help manage configuration, environments and externalized properties.

Key Idea

Make your application flexible to run in different environments with external configuration.

A. SPRING BOOT ANNOTATIONS

Annotation	Use	Example

@SpringBootApplication	Meta-annotation that enables auto-configuration, component scan & more.	`@SpringBootApplication`
		`public class MyApplication {`
		`  public static void main(String[] args) {`
		`  SpringApplication.run(MyApplication.class, args);`
		`  }`
		`}`

@EnableAutoConfiguration	Enables Spring Boot auto-configuration (usually used internally).	`@EnableAutoConfiguration`

@ConfigurationProperties	Binds external properties to a POJO.	`@Component`
		`@ConfigurationProperties(prefix = "app")`
		`public class AppProperties {`
		`  private String name;`
		`  private int port;`
		`  // getters & setters`
		`}`

Example - application.yml

```yaml
app:
  name: TomatoViews
  port: 8080

spring:
  datasource:
    url: jdbc:mysql://localhost:3306/db
    username: root
    password: root123
```

Note:

★ @ConfigurationProperties is a type-safe way to read all related properties with a prefix.

B. PROFILE ANNOTATIONS

Annotation	Use	Example

@Profile	Loads beans only for specific profiles.	`@Service`
		`@Profile("dev")`
		`public class DevService { }`

@ActiveProfiles	Used in test classes to activate profiles.	`@SpringBootTest`
		`@ActiveProfiles("test")`
		`class UserServiceTest { }`

application.properties example

```
# application-dev.properties
app.name = Dev App
app.port = 8081

# application-prod-properties
app.name = Prod App
app.port = 8080
```

How it works?

`@Profile("dev")` → Run with `--spring.profiles.active=dev` → Only Dev Beans will load

★ If no profile is specified, beans without @Profile will load.

@Value Examples

```
@Value("${db.url}")
private String dbUrl;

@Value("${db.timeout:30}")
private int timeout; // default 30

@Value("#{2 * 5}")
private int result; // SpEL

@Value("#{'hello' + 'world'}")
private String msg; // SpEL
```

C. PROPERTY & VALUE ANNOTATIONS

Annotation	Use	Example / Use

@Value	Injects values from properties files into variables.	`@Value("${app.name}") private String appName;`
		`@Value("${app.port:8080}") // default value`
		`private int port;`

@PropertySource	Loads properties file from custom location.	`@Configuration`
		`@PropertySource("classpath:custom.properties")`
		`public class AppConfig { }`

@EnableConfigurationProperties	Enables @ConfigurationProperties classes (when not using @SpringBootApplication).	`@SpringBootApplication`
		`@EnableConfigurationProperties(AppProperties.class)`
		`public class MyApplication { }`

@Import	Imports additional configuration classes.	`@Import(ExtraConfig.class)`
		`@Configuration`
		`public class MainConfig { }`



Pro Tip:

Prefer application.yml over .properties for better structure and readability.

10. TEST, VALIDATION & DOCUMENTATION ANNOTATIONS

→ These annotations help in testing, validation and API documentation.

Key Idea

Improve code quality, validate input/output and generate API docs with ease.

A. TESTING & VALIDATION ANNOTATIONS

Annotation	Use	Example
@Valid	Triggers validation on bean based on constraints (JSR-380).	@PostMapping("/create") public Response create(@Valid @RequestBody UserRequest request) { ... }
@NotNull	Field must not be null.	@NotNull(message = "Name cannot be null") private String name;
@NotBlank	Field must not be blank (used for String).	@NotBlank(message = "Username cannot be blank") private String username;
@Size	Validate size of collection or string.	@Size(min = 3, max = 10, message = "Name must be 3-10 characters") private String name;
@Pattern	Validate String with regular expression.	@Pattern(regexp = "^[A-Za-z0-9_]+$", message = "Invalid format") private String code;
@Email	Validate email format.	@Email(message = "Invalid email") private String email;

Common Use Case

- Validate user input
- Ensure correct data format
- Prevent bad requests
- Return meaningful error messages

Example - Validation in Controller

```
@PostMapping("/user")
public Response createUser(
   @Valid @RequestBody UserRequest req
) {
    // if validation fails, Spring
    // returns 400 Bad Request with
    // error details
}
```

B. DOCUMENTATION ANNOTATIONS (SpringDoc / OpenAPI)

Annotation	Use	Example
@Tag	Define group (tag) for API in documentation.	@Tag(name = "User Management", description = "APIs related to user operations")
@Operation	Summary of the API operation.	@Operation(summary = "Create new user", description = "Creates a new user in the system")
@ApiResponse	Define possible API responses.	@ApiResponse(responseCode = "201", description = "User created successfully")
@Parameter	Describe request parameters.	@Parameter(name = "id", description = "User ID", required = true)
@Schema	Describe model/schema in request/response.	@Schema(description = "User details", example = "Aman") private String name;

Benefits

- ★ Auto-generates interactive API documentation (Swagger UI).
- ★ Helps frontend & other teams understand APIs easily.
- ★ Improves maintainability and reduces communication gaps.

Example - OpenAPI Usage

```
@Tag(name = "User")
@Operation(summary = "Get user by ID")
@GetMapping("/user/{id}")
public User getUser(
    @Parameter(description = "ID of user",
    required = true) @PathVariable Long id) {
    ...
}
```

C. SUMMARY CHEAT SHEET

For Validation

- @Valid → Validate object
- @NotNull → Must not be null
- @NotBlank → Must not be blank
- @Size → Size constraints
- @Pattern → Regex validation
- @Email → Email format

For Documentation

- @Tag → Group APIs
- @Operation → Describe API
- @ApiResponse → Document responses
- @Parameter → Describe parameters
- @Schema → Describe models

Pro Tips

- ✓ Always use @Valid with request body.
- ✓ Provide meaningful messages in validations.
- ✓ Use Swagger annotations to keep API docs up-to-date with code.